Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Examples: Using the API to Retrieve Sensor Details

You can use the Stellar Cyber API to query the DP for a detailed list of its managed sensors.

Refer to **Configuring API Authentication** for general requirements to use the API.

API Syntax for Retrieving Sensor Details

The syntax for retrieving sensor details via the API is as follows:

`https://URL/connect/api/v1/data_sensor/<sensor_id>?cust_id=<tenant_id>`

Arguments:

- **<sensor_id>** – Optional. You can use this argument to query for details for a specific sensor using its ID retrieved from the **System | DATA SOURCE MANAGEMENT | Sensors | Sensors** page, as described below.

  If you do not include a **sensor_id**, the API returns all sensors belonging to the specified tenant (if **tenant_id** is specified) or the DP as a whole (if no **tenant_id** is specified).

- **<tenant_id>** – Optional. You can use this argument to query for all sensors assigned to a specified tenant using its ID retrieved from either **System | DATA SOURCE MANAGEMENT | Sensors | Sensors** or **System | ORGANIZATION MANAGEMENT | Tenants**, as described below.

  Note that if the sensor for which you are querying is assigned to a specific tenant, you **must** include the **tenant_id**.

Types of API Calls for Sensor Details

Depending on how you formulate the API call using the arguments listed above, you can make the following types of queries. Examples of each are provided following the summary below. Values in bold are the values you must supply as part of the query.

- Query for a single specified sensor.

Note that you only need to include the **tenant_id** if the specified sensor is assigned to a tenant:

`https://<Stellar Cyber_IP>/connect/api/v1/data_sensor/<sensor_id>?cust_id=<tenant_id>`

- Query for all sensors managed by the DP:

  `https://<Stellar Cyber_IP>/connect/api/v1/data_sensor`

- Query for all sensors belonging to a specific tenant:

  `https://URL/connect/api/v1/data_sensor?cust_id=<tenant_id>`

In response, the API returns detailed information on all sensors matching the API call. The information returned is similar to that presented in the System | Sensor page and is explained below in **Sensor Information Returned by the API**

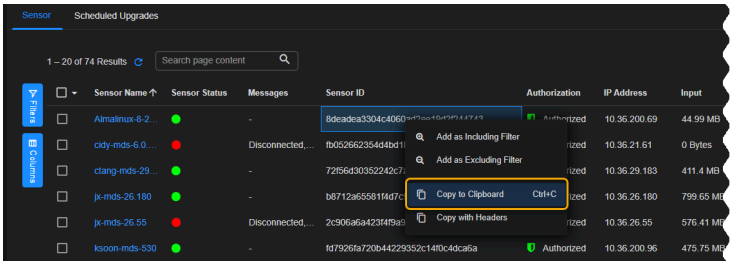
Finding the Sensor ID and Tenant ID

Several of the examples below require you to use the Sensor ID and Tenant ID in your query. You can find these values as follows:

1. Navigate to **System | DATA SOURCE MANAGEMENT | Sensors | Sensors**.
2. Select the **Change Columns** button and add **Tenant ID** to the display by selecting its box.

   Note that you can also find the Tenant ID in **System | ORGANIZATION MANAGEMENT | Tenants**. However, adding the column here lets you see the Sensor ID together with its corresponding Tenant ID.

3. Locate the table entry for the sensor for which you want to query.
4. Right-click the entry in the **Sensor ID** column and use the **Copy to Clipboard** command in the context menu that appears to copy the Sensor ID. For example:



5. Paste the Sensor ID into a text file to store it temporarily.
6. Right-click in the **Tenant ID** column and use the **Copy to Clipboard** command in the context menu that appears to copy the Tenant ID.
7. Paste the Tenant ID into a text file to store it temporarily.

Examples

The following sections provide examples of common ways to use the data_sensor API. The examples all use Python, though any programming language will work.

Querying for a Single Sensor

The following example uses a Python script to query for a single sensor with the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **API Key (Refresh Token) from GUI** – eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZXJfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYjQyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2SenlwUTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo
- **Sensor ID** – 48a1cea4e37641fc882a4a10c43b1a75
- **Tenant ID (cust_id)** – 7de229c6998041d1b84d9cb0e8893752

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the data_sensor API in the **getSensors** procedure (**Step 3** in the sample). The call includes the Sensor ID and Tenant ID for the desired sensor.
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

def getSensors(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST,
"/connect/api/v1/data_sensor/48a1cea4e37641fc882a4a10c43b1a75?
cust_id=7de229c6998041d1b84d9cb0e8893752", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":
```

```
    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt ------------")
    print(jwt)
    print("------------ jwt end ------------")

    # Step 3: use JWT token to call public API
    sensors = getSensors(jwt)
    print("------------ call result of /connect/api/v1/sensors ------------")
    print(sensors)
    print("------------ end api results ------------")
```

Querying for All Sensors Across All Tenants

The following example uses a Python script to query for all sensors across all tenants with the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **API Key (Refresh Token) from GUI** – eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZXJfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYjQyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2SenlwUTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo

Because this script queries for all sensors across all tenants, neither the Sensor ID nor the Tenant ID is included.

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the data_sensor API in the **getSensors** procedure (**Step 3** in the sample).
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
```

```
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

def getSensors(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/data_sensor", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt -------------")
    print(jwt)
    print("------------ jwt  end -------------")

    # Step 3: use JWT token to call public API
    sensors = getSensors(jwt)
    print("------------ call result of /connect/api/v1/sensors -------------")
    print(sensors)
    print("------------ end api results -------------")
```

Querying for a Single Tenant's Sensors

The following example uses a Python script to query for all sensors belonging to a specific tenant with the following details:

- **Stellar Cyber DP IP Address** – myserver.stellarcyber.cloud
- **API Key (Refresh Token) from GUI** – eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZXJfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYjQyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2SenlwUTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo
- **Tenant ID** – 7de229c6998041d1b84d9cb0e8893752

Understanding the Script

This script works as follows:

- The script sets the **host** and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the data_sensor API in the **getSensors** procedure (**Step 3** in the sample). The script includes the Tenant_ID in the URL.
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()
```

```
# Step 1
# Add DP IP/hostname, userid and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImY3NGYzYyIsInVzZX
JfaWQiOiI1YjdlNTA5ZjMyYmM0MmEzYWU5MTZiNDY0NzRmZGE0YyIsIm9yZ19pZCI6IjhlYTJiOWU3OTcxYj
QyZjZhNzJmZTYyODYyOTEwN2JmIiwiZXhwIjo0ODkyOTE0MzY3fSwiaWF0IjoxNzM5MzE0MzY3fQ._2Senlw
UTvmoAj4gFOehLyGtrKLRp8XfFrn8U35TOIo"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

def getSensors(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/data_sensor?
tenant_id=7de229c6998041d1b84d9cb0e8893752", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt -------------")
    print(jwt)
    print("------------ jwt  end -------------")

    # Step 3: use JWT token to call public API
    sensors = getSensors(jwt)
    print("------------ call result of /connect/api/v1/sensors -------------")
    print(sensors)
    print("------------ end api results -------------")
```

Sensor Information Returned by the API

The API returns the following information for each sensor matching the API call in field:value pairs, with fields separated from values by a colon. Separate field:value pairs are separated with commas for easy import. If a field:value pair has subvalues, they are listed inside pairs of backward slashes (for example, **\"timestamp\":1628282662**)refer to **Sample Output** for an example.

API Field Name	Description
total	If the query returned multiple sensors, the total is reported here.
_id	Internal ID for the sensor.
aggregator_list	Lists aggregator destinations for this sensor, if any.
additional_config	Lists the following additional sensor configuration: • **aella_asset** – Indicates whether asset detection is enabled for this sensor. Most sensors will show enabled; only sensors running legacy versions can toggle asset detection. • **aws_mirror** – Indicates whether the sensor is configured to receive traffic from an AWS mirror port. • **phy_nic** –The index number of the physical interface UDP traffic from an AWS mirror port arrives on. • **port** – The UDP port for traffic received from an AWS mirror port. • **VNI** – The VXLAN ID configured on AWS for mirrored traffic. • **tls_syslog** – Indicates whether the sensor is enabled to ingest TLS-encrypted logs. See Ingesting Logs Via TLS for details.
cpu_usage	The percentage CPU usage for the sensor at the time the data was pulled.
tenantid	The ID of the Tenant to which the sensor belongs, if any.

API Field Name	Description
cust_name	The Tenant Name to which this sensor belongs.
disk_usage	The percentage usage of the sensor's available disk space for the first disk on the sensor.
service_status	Lists the services running on the sensor. Services are listed with a 0 if they are running; all other values indicate the service is not running. The services listed depend on the sensor type and can include the following: • logfwd_out • logfwd_filter • logfwd_in • maltrace_out • svc_status • aella_mon • aella_ctrl • td-agent • maltrace • aella_savdid • suricata • aella_flow • aella_conf • logfwd_ext_server • aella_winlog • aella_diagnostics • winlog_out
nat_ip_address	If the sensor connects to the DP using network address translation (NAT), the NAT IP address observed by the DP is listed here.
sensor_id	The unique Sensor ID assigned to this sensor by the DP.
feature	Identifies the sensor type, as follows: • **ds** – data sensor • **sds** – security data sensor • **wds** – windows data sensor • **modular** – modular data sensor

feedback	Shows the last status messages sent from the sensor to the DP, if any. Includes information on upgrades, malware uploads, and Windows log file configuration. The same information is visible in the Sensor Overview by enabling the **Feedback** column and clicking the **Show Feedback** button. Here is a sample feedback section in the API output:
	`"feedback":"{\"upgrade\":{\"status\":\"Upgrade image successfully\"`
	`\"timestamp\":1628282662`
	`\"image\":\"aellads_4.1.1_ubuntu-16.04_20210805_19d6901.tar.gz\"}`
	`\"nessus-collector\":{\"msg\":\"Failed to login nessus security center\"`
	`\"status\":false`

API Field Name	Description
	\"timestamp\":4437277.39003} \"customer_malware_analyze\": {\"orig_name\":\"2320d0a07b733655e9de57bc1a11888736b7ad37d11f63e48f6552c10d4c3609\" \"task\":\"send-result\" \"status\":\"success\" \"timestamp\":\"2021-07-16T22:18:20Z\" \"file_name\":\"/var/log/cust_analyze/files/cust_file.3\" \"engine_id\":\"2932a8a3710140e2\" \"msg\":\"Analyze result is in sending\" \"tenant_id\":\"root_tenant\"}}"
hostname	The sensor's hostname.
internal_sensor_id	The internal ID of the sensor.
local_ip_address	The local IP address of the sensor.
license	The type of license for this sensor.
license_apt	Indicates whether the sensor is licensed for Advanced Persistent Threat (Malware Sandbox) features.
license_ids	Indicates whether the sensor is licensed for IDS features.
license_log	Indicates whether the sensor is licensed for log forwarding.
mem_usage	Displays the current percentage memory usage at the time the data was pulled from the sensor.
mode	Indicates whether the sensor is a physical sensor (**device**) or an **agent** installed on a host server.
module_version	Indicates the module version for a modular sensor.
need_upgrade	Indicates whether the sensor is eligible to be upgraded: Set to **true** if the sensor version is older than the DP version; otherwise, set to **false**.
os	The operating system on which the sensor is running.
platform	Indicates the type of virtual environment in which the sensor is deployed, including: • aws • azure • kvm • qemu • vmware
inbytes_total	The total number of bytes received by the sensor since the last time the aella_flow service restarted or the system rebooted.
sensor_profile_name	The sensor profile assigned to the sensor.
auth_state_code	Indicates whether the sensor has been authorized from the user interface.

API Field Name	Description
connection_status	Indicates whether the sensor is currently **connected** to or **disconnected** from the DP.
sw_version	The version of Stellar Cyber software running on the sensor.
timezone	The time zone configured for the sensor. During installation, the timezone for sensors are automatically set to UTC+0. Since the logs for some security products might only include the local time without a timezone, Stellar Cyber recommends that you set the sensor timezone to the same timezone as your security product.
last_stats_time	The timestamp at which the sensor uploaded statistics to the DP, expressed in epoch time.
tunnel_enabled	Indicates whether a reverse SSH tunnel is enabled between this sensor and the DP.
tunnel_info	reverse ssh tunnel information
outbytes_total	The total number of bytes transmitted by the sensor since the last time the aella_flow service restarted or the system rebooted.
packet_forwarding_interface	The VXLAN interface number of the packet forwarding interface.
cm_worker_id	The numerical ID of the CM worker node to which the sensor is connected.
message	The latest status message associated with this sensor, if any. Messages appear in the Sensor List in the Messages column, if displayed.

Sample Output

The text below shows sample output of an API call for a single sensor's information. In this case, the sensor is a Server Sensor and does not have some of the fields that a device sensor would (for example, **cm_worker_id** or **packet_forwarding_interface**)


```
{'_id': '6577d1076333ceb1dcbd8414', 'aggregator_list': '[]', 'cpu_usage':
11.9, 'tenantid': '1078ceaaa1644e84b1478a5a4947c09c', 'cust_name': 'hq1',
'disk_usage': 17.9, 'service_status': '{"svc_status": {"aella_flow": 0,
"aella_mon": 0, "aella_conf": 0, "aella_ctrl": 0, "aella_audit": 0},
"audit_out": 2646362}', 'nat_ip_address': '205.234.30.196', 'sensor_id':
'1454ef82de9347a49ca196f107317cac', 'feature': 'ds', 'feedback': '{"upgrade":
{"status": "Upgrade image successfully", "result": "succeed", "engine_id":
"1454ef82de9347a49ca196f107317cac", "tenant_id":
"1078ceaaa1644e84b1478a5a4947c09c", "timestamp": 1702367228, "image":
"aellads_5.1.0_redhat-binary_20231212_5a6221c.tar.gz", "msg_id": 2}}',
'hostname': 'almalinux-9', 'internal_sensor_id':
'1454ef82de9347a49ca196f107317cac', 'local_ip_address': '10.33.5.249',
'license': 'unlimited', 'license_log': True, 'mode': 'agent',
'module_version': '{"log_forwarder": ""}', 'os': 'almalinux-9', 'platform':
'vmware', 'inbytes_total': 22599805, 'sensor_profile_name': 'Default',
'auth_state_code': 1001, 'connection_status': 'connected', 'sw_version':
'5.1.0_5a6221c', 'last_stats_time': 1702379371150, 'outbytes_total': 3655908,
'message': '', 'need_upgrade': False}
```

© 2026 Stellar Cyber . All rights reserved.